Interrogation Avancée en SQL : Jointures Relationnelles et Regroupements (GROUP BY & HAVING)

Thème principal : Interrogation de bases de données relationnelles multi-tables et calculs d'agrégats groupés

Synthèse exécutive : Ce cours fondamental de la ressource R1.05 détaille le fonctionnement interne de l'interrogation SQL complexe. Il couvre l'association de données réparties dans plusieurs tables grâce aux jointures relationnelles (INNER JOIN, LEFT JOIN) et la manipulation statistique des données à l'aide des fonctions d'agrégation, associées aux clauses GROUP BY et HAVING. L'accent est mis sur la logique d'exécution séquentielle d'une requête SQL par le SGBD afin d'éviter les produits cartésiens involontaires et d'assimiler la distinction primordiale entre le filtrage de lignes (WHERE) et le filtrage de groupes (HAVING).

1. Les Jointures Relationnelles : Interrogation Multi-Tables (INNER JOIN vs LEFT JOIN)

Dans un Système de Gestion de Base de Données Relationnel (SGBDR), les données ne sont jamais stockées dans un tableau unique afin d'éviter les anomalies de mise à jour et la redondance d'information. La normalisation répartit les concepts métiers dans des tables distinctes. La jointure est l'opération algébrique indispensable pour recomposer les informations au moment de la requête. Lors de l'évaluation du bloc FROM, le moteur de base de données associe les tables selon l'attribut pivot (généralement la clé étrangère d'une table pointant vers la clé primaire d'une autre). L'utilisation explicite de la clause INNER JOIN garantit que seules les paires valides apparaîtront dans le résultat. En revanche, si l'on souhaite procéder à un inventaire exhaustif sans perdre les éléments sans correspondance (par exemple, tous les étudiants, y compris ceux sans notes), le LEFT JOIN est indispensable : les attributs manquants de la table de droite sont alors comblés par des valeurs 'NULL'. Omis ou mal configuré, le prédicat de liaison génère un produit cartésien très lourd qui surcharge la mémoire et le processeur du SGBD.

- Normalisation & Équijointure : Séparation logique des entités pour éliminer les redondances et réassociation via la Clé Primaire (PK) et la Clé Étrangère (FK).
- INNER JOIN (Jointure Interne) : Restitue exclusivement les combinaisons de n-uplets qui satisfont la condition d'équivalence définie dans la clause ON.
- LEFT JOIN (Jointure Externe Gauche) : Conserve la totalité des enregistrements de la table de gauche, complétés par la valeur NULL si aucune correspondance n'existe dans la table de droite.
- Produit Cartésien (CROSS JOIN) : Résultat de l'omission involontaire du critère de jointure (ON), combinant chaque ligne de la table A avec toutes les lignes de la table B (N x M n-uplets).

Cas concret enseignant : *Le professeur illustre la différence à travers deux cas réels de l'IUT de Bayonne : 1. L'INNER JOIN fonctionne comme le portique de sécurité du campus : seuls les étudiants possédant un badge actif ET une inscription valide peuvent franchir la porte. Si l'un des deux éléments manque, l'accès est refusé (la ligne est exclue). 2. Le LEFT JOIN équivaut à la liste d'appel officielle du secrétariat : tous les étudiants inscrits sur le registre de l'IUT y figurent obligatoirement. Si un étudiant n'a pas encore de note attribuée, son nom reste affiché et la case note contient la mention 'NULL'.*

2. Regroupement de Données et Agrégation : Clauses GROUP BY et HAVING

L'agrégation permet de répondre à des problématiques décisionnelles (calcul de moyennes, sommes de ventes, décomptes d'effectifs). Dès qu'un regroupement est initié via la clause GROUP BY, la table d'origine se retrouve partitionnée en sous-ensembles disjoints. Une erreur courante en évaluation consiste à tenter de filtrer un calcul statistique dans la clause WHERE (par exemple 'WHERE AVG(note) > 10'). Cette écriture est physiquement rejetée par le processeur de requêtes SQL. En effet, la clause WHERE est évaluée ligne par ligne au niveau de la table initiale, BIEN AVANT que les groupes ne soient formés et que les fonctions d'agrégation ne soient calculées. Pour appliquer un critère de restriction sur le résultat d'un calcul agrégé, SQL met à disposition la clause HAVING, qui

intervient exclusivement APRÈS la phase de regroupement des tuples.

• Fonctions d'Agrégation (COUNT, SUM, AVG, MIN, MAX) : Calculs statistiques scalaires réduisant un ensemble de n-uplets à une valeur synthétique unique.

• Clause GROUP BY : Subdivise les n-uplets filtrés par la clause WHERE en sous-groupes homogènes partageant les mêmes valeurs pour les colonnes spécifiées.

• Règle d'or du SELECT avec GROUP BY : Toute colonne présente dans la clause SELECT doit obligatoirement figurer dans la clause GROUP BY ou être intégrée dans une fonction d'agrégation.

• Clause HAVING : Prédicat de filtrage s'appliquant spécifiquement sur les sous-groupes formés et sur le résultat des fonctions agrégées.

• Ordre logique d'exécution SQL : FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY.

Cas concret enseignant : *L'enseignant prend l'exemple du dépouillement physique des copies d'examen dans l'amphithéâtre de l'IUT : 1. GROUP BY : Le professeur répartit toutes les copies physiques dans trois bannettes distinctes selon le département (Informatique, GEA, TC). 2. Agrégation (AVG) : Pour chaque bannette, il calcule la note moyenne du groupe. 3. HAVING : Si le directeur impose d'afficher uniquement les paquets dont la moyenne globale dépasse 12/20, c'est la clause HAVING qui élimine les bannettes non conformes. La clause WHERE, elle, aurait servi avant le tri à jeter les copies d'étudiants fraudeurs.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi est-il strictement impossible d'écrire 'WHERE AVG(note) > 10' dans une requête SQL ?
Réponse du professeur : La clause WHERE est évaluée par le moteur SQL très tôt dans le cycle d'exécution, au moment du parcours ligne par ligne des tables du FROM. À ce stade, le regroupement n'a pas encore eu lieu et la moyenne n'existe pas encore. Pour filtrer sur le résultat d'une fonction d'agrégation comme AVG(), il faut obligatoirement utiliser la clause HAVING, qui s'exécute après le GROUP BY.

Question étudiant : Que se passe-t-il si j'oublie d'inclure dans la clause GROUP BY une colonne présente dans le SELECT ?
Réponse du professeur : Le SGBD (en mode SQL strict ou sous PostgreSQL/Oracle) lève une erreur de syntaxe immédiate : 'Column must appear in the GROUP BY clause or be used in an aggregate function'. C'est une règle absolue : si une colonne n'est ni la clé de regroupement ni contractée par une fonction d'agrégation (comme SUM ou AVG), le SGBD ne sait pas quelle valeur individuelle afficher parmi les multiples lignes du groupe.

■ Exercice & Démonstration d'Application

Énoncé : Soit la base de données relationnelle de l'IUT composée des tables : - Etudiant(id_etud, nom, id_promo) - Note(id_note, id_etud, matiere, note) Écrire la requête SQL qui affiche le nom de chaque étudiant et sa moyenne dans la matière 'R1.05', uniquement pour les étudiants ayant au moins 2 notes enregistrées dans cette matière et une moyenne stricte supérieure à 12/20. Le résultat doit être trié par moyenne décroissante.

Résolution pas à pas :
1. Identifier les tables et réaliser la jointure interne dans le FROM : FROM Etudiant E INNER JOIN Note N ON E.id_etud = N.id_etud. 2. Filtrer les enregistrements individuels pour ne conserver que la matière 'R1.05' via le WHERE : WHERE N.matiere = 'R1.05'. 3. Regrouper les données par identifiant et par nom d'étudiant avec GROUP BY : GROUP BY E.id_etud, E.nom. 4. Filtrer les groupes d'étudiants respectant les deux conditions d'agrégation dans le HAVING : HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12. 5. Définir la projection de sortie et nommer l'agrégat dans le SELECT : SELECT E.nom, AVG(N.note) AS moyenne. 6. Trier le résultat final par ordre décroissant de la moyenne via le ORDER BY : ORDER BY moyenne DESC.

Résultat : SELECT E.nom, AVG(N.note) AS moyenne FROM Etudiant E INNER JOIN Note N ON E.id_etud = N.id_etud WHERE N.matiere = 'R1.05' GROUP BY E.id_etud, E.nom HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12 ORDER BY moyenne DESC;

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale entre la clause WHERE et la clause HAVING dans une requête SQL ?

R : La clause WHERE filtre les lignes individuelles avant tout regroupement ou calcul d'agrégat. La clause HAVING filtre les sous-groupes de données après l'application de la clause GROUP BY et prend en charge les fonctions d'agrégation.

Q : Quel est le comportement d'un LEFT JOIN si une ligne de la table de gauche ne possède aucune correspondance dans la table de droite ?

R : La ligne de la table de gauche est conservée dans le jeu de résultats final, et tous les attributs provenant de la table de droite sont renseignés avec la valeur NULL.

Q : Si la table 'Etudiant' comporte 10 lignes et la table 'Note' comporte 5 lignes, combien de lignes retourne un produit cartésien (CROSS JOIN) entre ces deux tables ?

R : Le produit cartésien multiplie chaque ligne de la première table par toutes les lignes de la seconde, produisant ainsi 10 x 5 = 50 lignes.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765